

# MEDS Lab - Module 4

## Section 8 Report: Multiplexers & Shifters

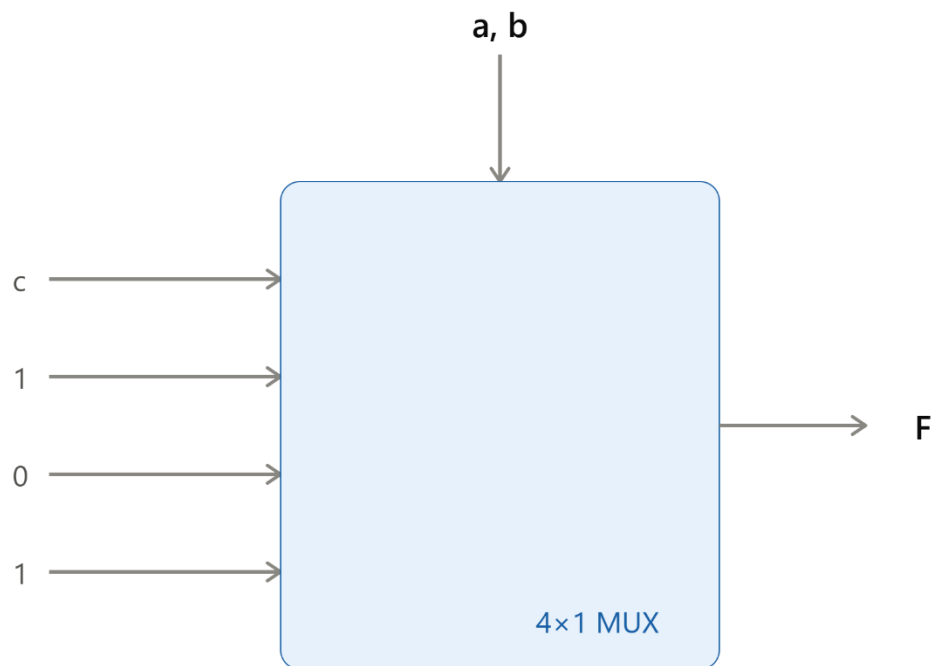
Name: Muhammad Kaleem Faryad

University: UET Lahore

### Task 8(v1): $F = \Sigma m(1,2,3,6,7)$ Using a 4x1 MUX

1. EDA Playground Link: <https://edaplayground.com/x/MmsK>

2. Block Diagram:



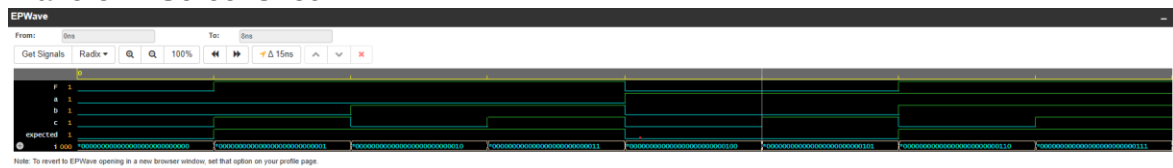
### 3. Simulation Output Screenshot:

The screenshot displays a simulation output window with two main sections: a code editor on the left and a simulation report on the right.

**Code Editor (Left):** Shows Verilog code for a testbench and a 4-to-1 multiplexer. The testbench code includes a task for automatic checking, a loop for simulation, and a final display statement. The multiplexer code defines a module with inputs a, b, c and output y, and a function to select the output based on the inputs.

**Simulation Report (Right):** Contains a log of simulation results, including the compiler version (X-2025.06-SP1\_Full164), runtime version (X-2025.06-SP1\_Full164), and a summary of the simulation results. The report indicates that the simulation was successful and that the output y was 1 for all input combinations of a and b.

### 4. Waveform Screenshot:



- Explanation:** The objective of this task was to implement the Boolean function  $F = \sum m(1, 2, 3, 6, 7)$  using a **4-to-1 Multiplexer (MUX)**. The first step was to expand the minterm notation into a complete truth table for the three input variables **a**, **b**, and **c**. The variables **a** and **b** were selected as the multiplexer select lines because a 4-to-1 MUX requires two select inputs. This divided the truth table into four groups corresponding to the four possible values of **ab** (00, 01, 10, and 11). For each value of **ab**, the output was observed as **c** changed from 0 to 1. Depending on the output pattern, each MUX data input was replaced with one of four possible values: Constant **0**, Constant **1**, Variable **c**, Complement **c'**. The derived data inputs were:

| Select (ab) | Data Input |
|-------------|------------|
| 00          | c          |
| 01          | 1          |
| 10          | 0          |
| 11          | 1          |

- These values were connected to the inputs of the 4-to-1 multiplexer, while **a** and **b** were connected to the select lines. The multiplexer automatically selected the

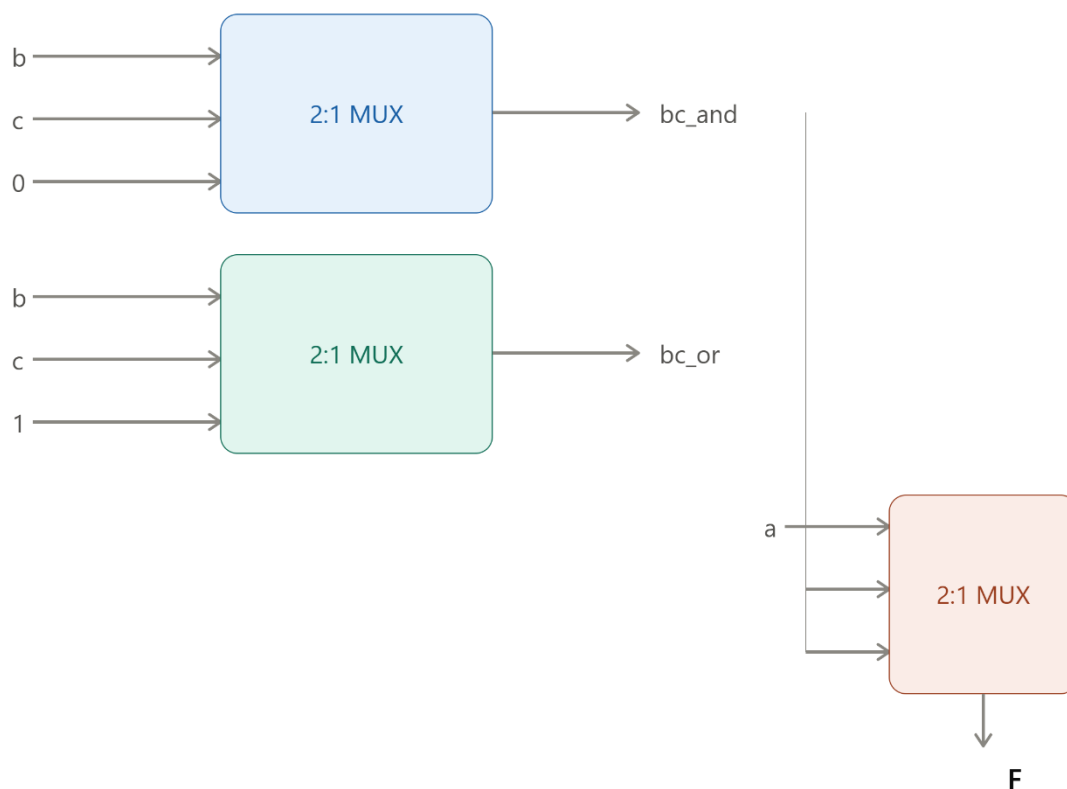
correct data input according to the values of **a** and **b**, producing the required Boolean function.

- The testbench applied all eight possible input combinations (000–111) and compared the DUT output with the expected truth table. Every test passed successfully, confirming that the MUX implementation correctly realizes the given Boolean function.

## Task 8(v2): Extension of Task 8 (majority function)

1. **EDA Playground Link:** <https://edaplayground.com/x/j7xs>

2. **Block Diagram:**



### 3. Simulation Output Screenshot:

```

// Code your testbench here
// or browse examples
module tb;
    logic a,b,c;
    logic f;

    majority_mux dut0(
        .a(a),.b(b),.c(c),.f(f)
    );

    logic expected;

    task automatic check;
        if(f !== expected)
            $display("[FAIL] a=%0b b=%0b c=%0b\n",a,b,c,expected,f);
        else
            $display("[PASS] a=%0b b=%0b c=%0b\n",a,b,c,f);
    endtask

    integer i;
    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(0,tb);
        for(i=0;i<8;i++)begin
            {a,b,c}=i;
            #1;
            check();
        end
    $finish;
endmodule
    
```

```

module majority_mux(
    input logic a,b,c,
    output logic f
);
    logic and_bc;
    logic or_bc;

    // b & c
    //when b = 0 => and_bc = 0
    //when b = 1 => and_bc = c
    mux2to1 dut1(
        .sel(b),.d0(1'b0),.d1(c),.y(and_bc)
    );

    // b | c
    //when b=0 => or_bc = c
    //when b=1 => or_bc = 1
    mux2to1 dut2(
        .sel(b),.d0(c),.d1(1'b1),.y(or_bc)
    );

    //final MUX
    //when a = 0 => f=and_bc
    //when a = 1 => f=or_bc
    mux2to1 dut3(
        .sel(a),.d0(and_bc),.d1(or_bc),.y(f)
    );
endmodule
    
```

Compiler version X-2025.06-SP1\_Full164; Runtime version X-2025.06-SP1\_Full164; Jul 21 05:25 2026  
 [PASS] a=0,b=0 c=0 output=0  
 [PASS] a=0,b=0 c=1 output=0  
 [PASS] a=0,b=1 c=0 output=0  
 [PASS] a=0,b=1 c=1 output=1  
 [PASS] a=1,b=0 c=0 output=0  
 [PASS] a=1,b=0 c=1 output=1  
 [PASS] a=1,b=1 c=0 output=1  
 [PASS] a=1,b=1 c=1 output=1  
 \$finish called from file "testbench.sv", line 32.  
 \$finish at simulation time 8  
 V C S S I M U L A T I O N R E P O R T  
 Time: 8 ns  
 CPU Time: 0.530 seconds; Data structure size: 0.0Mb  
 Tue Jul 21 05:25:20 2026

### 4. Waveform Screenshot:



- Explanation:** The extension implements the **3-bit Majority Function** using only **cascaded 2-to-1 multiplexers**. A majority function outputs **1** when at least **two of the three inputs (a, b, c)** are 1, and is given by:

$$F = ab + ac + bc$$

Instead of using AND and OR gates directly, the design uses only multiplexers. By selecting **a** as the control signal of the final 2-to-1 MUX, the truth table simplifies to:

**F = b & c** when **a = 0**, and

**F = b | c** when **a = 1**.

Thus, the final MUX selects between these two expressions based on the value of **a**.

The **AND** operation (**b & c**) is implemented using one 2-to-1 MUX with **b** as the select input, **0** connected to **d0**, and **c** connected to **d1**.

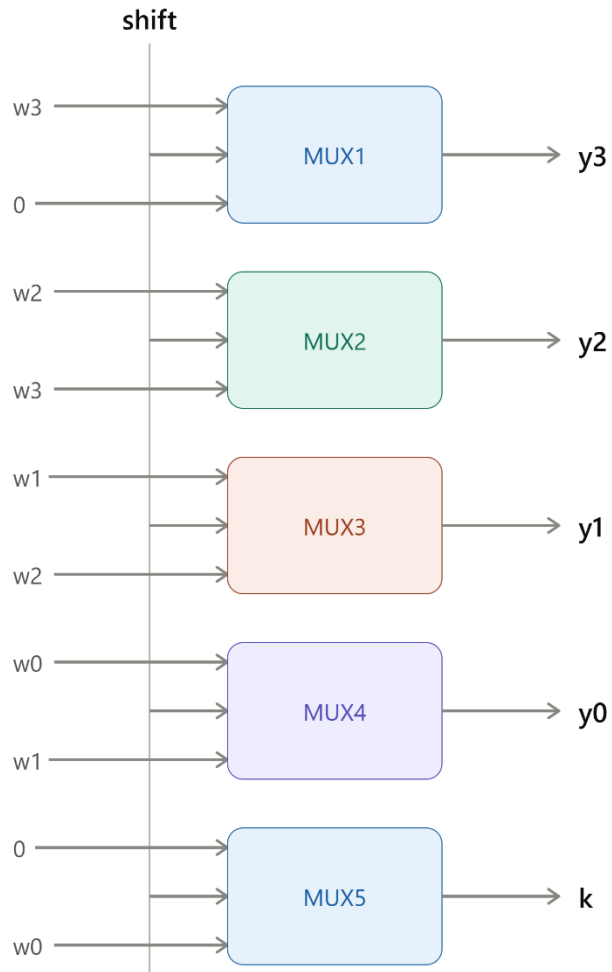
Similarly, the **OR** operation (**b | c**) is implemented using another 2-to-1 MUX with **b** as the select input, **c** connected to **d0**, and constant **1** connected to **d1**. A third MUX combines these two outputs to generate the final majority function.

The testbench exhaustively verified all **8 input combinations** against the expected expression **(a & b) | (a & c) | (b & c)**. All test cases passed successfully, confirming that the cascaded 2-to-1 MUX implementation correctly realizes the majority function.

## Task 9(PART\_A): 4-bit Barrel Shifter (single-position shift)

1. EDA Playground Link: <https://edaplayground.com/x/ejeL>

2. Block Diagram:



### 3. Simulation Output Screenshot:

```
// Code your testbench here
// or browse Examples
module tb;
    logic shift,w3,w2,w1,w0;
    logic y3,y2,y1,y0,k;

    logic exp_y3,exp_y2,exp_y1,exp_y0;
    logic exp_k;

    logic [3:0] W;

    singlebit_shift DUT(
        .shift(shift),.w3(w3),.w2(w2),.w1(w1),.w0(w0),
        .y3(y3),.y2(y2),.y1(y1),.y0(y0),
        .k(k)
    );

    task automatic check;
    begin
        if ((y3 == exp_y3) || (y2 == exp_y2) || (y1 == exp_y1) || (y0 == exp_y0) || (k == exp_k)) begin
            $display("[PASS] %4b shift=%0b k=%0b",W,shift,exp_y3,exp_y2,exp_y1,exp_y0,k);
        end
        else begin
            $display("[FAIL] %4b shift=%0b k=%0b",W,shift,exp_y3,exp_y2,exp_y1,exp_y0,k);
        end
    end
endmodule

// 2-to-1 Multiplexer
module mux2x1;
    input logic sel;
    input logic d0,d1;
    output logic y;

    assign y = sel ? d1 : d0;
endmodule

// Part A: Single Bit Right Shifter
module singlebit_shift;
    input logic shift;
    input logic w3,w2,w1,w0;
    output logic y3,y2,y1,y0,k;

    // MUX 1
    mux2x1 DUT1(
        .sel(shift),
        .d0(w3),
        .d1(y3)
    );

    // MUX 2
    mux2x1 DUT2(
        .sel(shift),
        .d0(w2),
        .d1(w3)
    );

    // MUX 3
    mux2x1 DUT3(
        .sel(shift),
        .d0(w1),
        .d1(w2)
    );

    // MUX 4
    mux2x1 DUT4(
        .sel(shift),
        .d0(w0),
        .d1(w1)
    );

    k = y3;
    y3 = y2;
    y2 = y1;
    y1 = y0;
    y0 = k;
endmodule
```

Compiler version X-2025.06-SPI\_Full164; Runtime version X-2025.06-SPI\_Full164; Jul 21 07:19 2026

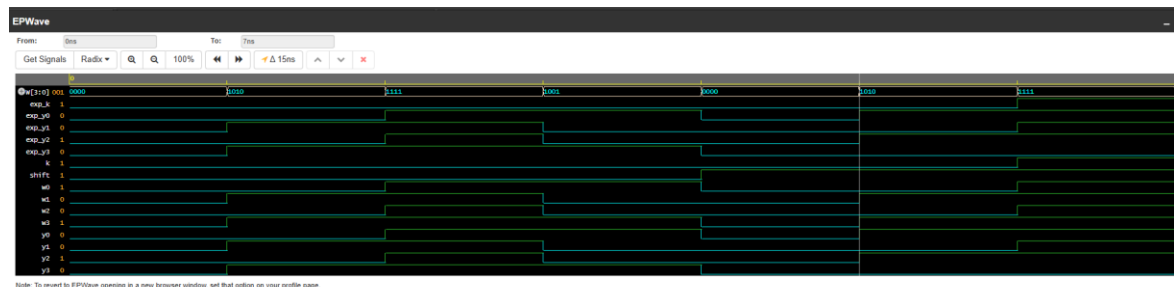
[PASS] W=0000 shift=0 Y=0000 K=0  
[PASS] W=1010 shift=0 Y=1010 K=0  
[PASS] W=1111 shift=0 Y=1111 K=0  
[PASS] W=1001 shift=0 Y=1001 K=0  
[PASS] W=0000 shift=1 Y=0000 K=0  
[PASS] W=1010 shift=1 Y=0101 K=0  
[PASS] W=1111 shift=1 Y=0111 K=1  
[PASS] W=1001 shift=1 Y=0100 K=1

\$finish called from file "testbench.sv", time 110.  
\$finish at simulation time 8

VCS Simulation Report

Time: 8 ns  
CPU Time: 0.520 seconds; Data structure size: 0.0Mb  
Tue Jul 21 07:19:42 2026  
Finding VCD file...  
./dump.vcd  
[2024-07-21 11:19:42 UTC] Opening EPWave...

### 4. Waveform Screenshot:

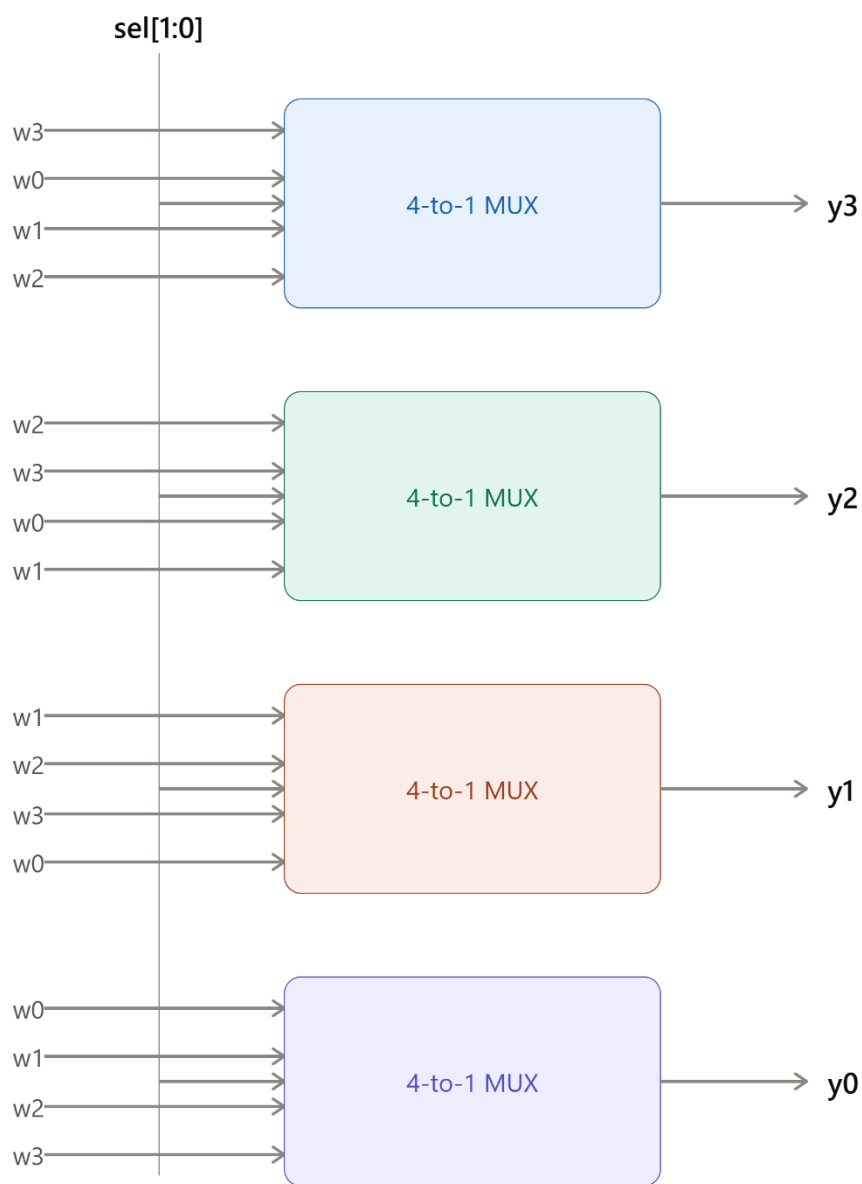


5. **Explanation:** In this task, a 4-bit fixed right shifter was designed using only **2-to-1 multiplexers**, implemented through conditional (MUX) logic as required. The circuit has a 4-bit input **W = w3w2w1w0**, a control signal **shift**, a 4-bit output **Y**, and a shifted-out bit **k**.
- When **shift = 0**, the input is passed directly to the output without any change, so **Y = W** and **k = 0**. When **shift = 1**, the input is shifted one position to the right. The most significant bit (**y3**) is filled with **0**, the remaining output bits receive the values of their left neighbours (**y2 = w3**, **y1 = w2**, **y0 = w1**), and the least significant bit (**w0**) is shifted out as the output **k**. The circuit was implemented using **five 2-to-1 multiplexers**, where each multiplexer selects between the normal value and the shifted value depending on the **shift** control signal. This implementation demonstrates how multiplexers can be used to perform data movement operations without using case statements or sequential logic. A self-checking testbench was developed to verify the design. Multiple input patterns were applied for both **shift = 0** and **shift = 1**, and the actual outputs were compared with the expected results. All test cases passed successfully, confirming that the fixed right shifter operates according to the specified behavior.

## Task 9(Part\_B): Extension of Task 9 (barrel shifter)

1. EDA Playground Link: <https://edaplayground.com/x/gGLy>

2. Block Diagram:



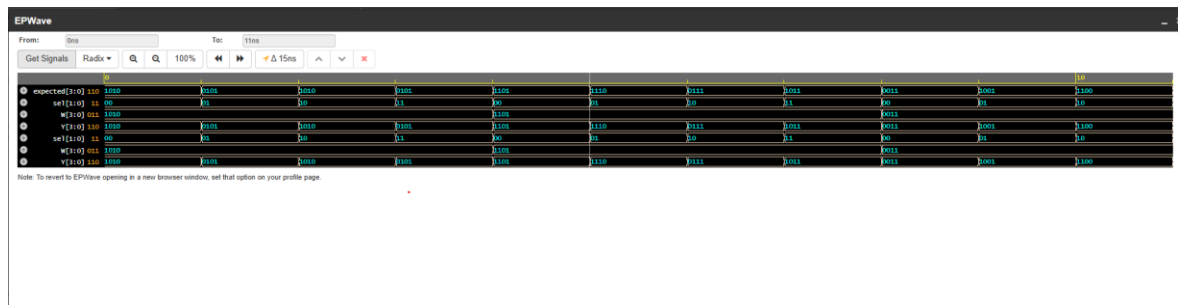
### 3. Simulation Output Screenshot:

```
1 // Code your testbench here
2 // or browse Examples
3 module tb;
4
5 logic [3:0] W;
6 logic [1:0] sel;
7
8 logic [3:0] Y;
9 logic [3:0] expected;
10
11 // DUT
12 barrel_shifter4 dut(
13     .W(W),
14     .sel(sel),
15     .Y(Y)
16 );
17
18 task automatic check;
19 begin
20     if (Y != expected)
21         $display("FAIL: W=%b sel=%b Z=%b", W, sel, Y);
22     else
23         $display("PASS: W=%b sel=%b Z=%b", W, sel, Y);
24 end
25
26 output = "W sel Y";
27
28 // Testbench
29 endmodule
```

```
1 // Code your design here
2 // Task 9 Part: B
3 // 4-bit Barrel Shifter (Rotate Right)
4
5 module barrel_shifter4(
6     input logic [3:0] W,
7     input logic [1:0] sel,
8     output logic [3:0] Y
9 );
10
11 always_comb
12 begin
13     unique case(sel)
14         // Rotate by 0
15         2'b00: Y = W;
16         // Rotate by 1
17         2'b01: Y = {W[0], W[3], W[2], W[1]};
18         // Rotate by 2
19         2'b10: Y = {W[1], W[0], W[3], W[2]};
20         // Rotate by 3
21         2'b11: Y = {W[2], W[1], W[0], W[3]};
22         default:
23             Y = 4'b0000;
24     endcase
25 end
26 endmodule
```

```
..$simv up to date
CPU time: .615 seconds to compile + .502 seconds to elab + .490 seconds to link
Chronologic VCS simulator copyright 1991-2025
Contains Synopsys proprietary information.
Compiler version X-2025.06-SP1_Fu1164; Runtime version X-2025.06-SP1_Fu1164; Jul 21 14:28 2026
[PASS] W=1010 sel=00 Output=1010
[PASS] W=1010 sel=01 Output=0101
[PASS] W=1010 sel=10 Output=1010
[PASS] W=1010 sel=11 Output=0101
[PASS] W=1101 sel=00 Output=1101
[PASS] W=1101 sel=01 Output=1110
[PASS] W=1101 sel=10 Output=0111
[PASS] W=1101 sel=11 Output=1011
[PASS] W=0011 sel=00 Output=0011
[PASS] W=0011 sel=01 Output=1001
[PASS] W=0011 sel=10 Output=1100
[PASS] W=0011 sel=11 Output=0110
$finish called from file "testbench.sv", time 100.
$finish at simulation time 12
```

### 4. Waveform Screenshot:



5. **Explanation:** The 4-bit barrel shifter was designed to perform a **circular right rotation** of the input word  $W = w_3w_2w_1w_0$  by **0, 1, 2, or 3 positions**, as selected by the 2-bit control signal **sel**. Unlike a normal shifter, a barrel shifter does not discard bits; instead, the bits shifted out from the least significant position are wrapped around and inserted into the most significant position.

The design was implemented using an `always_comb` block with a **unique case** statement. For `sel = 00`, the output is identical to the input. For `sel = 01`, the input is rotated right by one position. For `sel = 10`, it is rotated by two positions, and for `sel = 11`, it is rotated by three positions. Each case assigns the corresponding rotated arrangement of the input bits to the output.

A `unique case` statement was used instead of a normal `case` because only one value of `sel` can be valid at any time. This informs the simulator and synthesizer that exactly one branch should match, enabling additional simulation checks for missing or overlapping cases and allowing better hardware optimization.

A self-checking testbench was developed to verify the design. Three different input values of  $W$  were applied, and each was tested with all four values of `sel`. The actual output was compared with the expected rotated value, and every test case produced a **PASS** result, confirming that the barrel shifter performs the required rotation correctly for all specified operations.